Java Streams

Complete Notes

Beginner → Advanced

A reference guide for Java Stream API fundamentals,
Spring Boot development, and interview preparation

Table of Contents

1. What is Stream API?

A Stream is a sequence of elements that supports functional-style operations on data.

- Introduced in Java 8
- Processes data, doesn't store it
- Lazy by default
- Can only be consumed once
- Supports sequential and parallel execution

Think of it like a pipeline:

```
Collection
    |
    ▼
 Stream
    |
Intermediate Operations
    |
    ▼
Terminal Operation
    |
    ▼
Result
```

Example

```
List<String> names = List.of("Ram","Shyam","Amit");

names.stream()
     .filter(name -> name.startsWith("A"))
     .forEach(System.out::println);
```

2. Collection vs Stream

Collection	Stream
Stores data	Processes data
Mutable	Immutable view
Can iterate multiple times	Can iterate once
Eager	Lazy

3. Stream Lifecycle

```
Source
 ↓
Intermediate Operations
```

```
 ↓
Terminal Operation
 ↓
Result
```

Example

```
employees.stream()
         .filter(Employee::isActive)
         .map(Employee::getName)
         .sorted()
         .toList();
```

4. Creating Streams

From List

```
list.stream();
```

From Array

```
Arrays.stream(array);
```

Using Stream.of()

```
Stream.of(1,2,3);
```

Infinite Stream

```
Stream.generate(() -> "Hello");
```

Empty Stream

```
Stream.empty();
```

5. Intermediate Operations

These return another Stream.

filter()

Removes unwanted elements.

```
.filter(e -> e.getSalary() > 50000)
```

Input:

```
10 20 30 40
```

Output:

```
30 40
```

map()

Transforms one object into another.

```
.map(Employee::getName)

Employee
  ↓
Name
```

flatMap()

Flattens nested collections.

```
employees.stream()
         .flatMap(emp -> emp.getSkills().stream())
```

Without flatMap:

```
[[Java, Spring], [React]]
```

With flatMap:

```
Java
Spring
React
```

distinct()

Removes duplicates.

```
.distinct()
```

Uses:

- equals()
- hashCode()

sorted()

Ascending:

```
.sorted()
```

Descending:

```
.sorted(Comparator.reverseOrder())
```

By object field:

```
.sorted(
  Comparator.comparing(Employee::getSalary)
)
```

peek()

Used for debugging.

```
.peek(System.out::println)
```

Avoid modifying objects inside peek().

limit()

```
.limit(10)
```

Returns first 10 elements.

skip()

```
.skip(20)
```

Useful for pagination.

6. Terminal Operations

forEach()

```
.forEach(System.out::println)
```

Side effects only.

toList()

```
.toList()
```

Java 16+

collect()

```
.collect(Collectors.toList())
```

Creates:

- List
- Set
- Map

count()

```
.count()
```

Returns long.

findFirst()

Returns first element.

```
.findFirst()
```

```
.orElse(null)
```

findAny()

Returns any matching element. Better for parallel streams.

min()

```
.min(Comparator.comparing(Employee::getSalary))
```

max()

```
.max(...)
```

anyMatch()

At least one matches.

```
.anyMatch(e -> e.getSalary()>100000)
```

allMatch()

All elements satisfy condition.

noneMatch()

No element satisfies condition.

reduce()

Combines many values into one.

Sum:

```
.reduce(0,Integer::sum)
```

Maximum:

```
.reduce(Integer.MIN_VALUE,Integer::max)
```

7. Collectors

Import:

```
import java.util.stream.Collectors;
```

toList()

```
Collectors.toList()
```

toSet()

```
Collectors.toSet()
```

Removes duplicates.

toMap()

```
Collectors.toMap(
  Employee::getId,
  Employee::getName
)
```

Duplicate key handling:

```
Collectors.toMap(
  key,
  value,
  (old,newValue)->old
)
```

joining()

```
Collectors.joining(", ")
```

Output:

```
Ram, Shyam, Amit
```

groupingBy()

Most important collector.

```
Collectors.groupingBy(
  Employee::getDepartment
)
```

Output:

```
IT
  Manish
  Rahul

HR
  Amit
```

counting()

```
Collectors.counting()
```

Example:

```
Collectors.groupingBy(
  Employee::getDepartment,
  Collectors.counting()
)
```

mapping()

```
Collectors.mapping(
  Employee::getName,
  Collectors.toList()
)
```

Group employees but store only names.

partitioningBy()

Creates exactly two groups.

```
Collectors.partitioningBy(
  e->e.getSalary()>100000
)
```

Result:

- true
- false

averagingDouble()

```
Collectors.averagingDouble(
  Employee::getSalary
)
```

summingDouble()

```
Collectors.summingDouble(
  Employee::getSalary
)
```

summarizingInt()

Returns:

- Count
- Sum
- Average
- Min
- Max

```
Collectors.summarizingInt(
  Employee::getAge
)
```

maxBy()

```
Collectors.maxBy(...)
```

minBy()

```
Collectors.minBy(...)
```

8. Parallel Stream

Sequential:

```
list.stream()
```

Parallel:

```
list.parallelStream()
```

Uses:

```
ForkJoinPool.commonPool()
```

Good for:

- Large data
- CPU intensive work

Avoid for:

- Database queries
- REST API calls
- File IO
- Small collections

9. Primitive Streams

- IntStream
- LongStream
- DoubleStream

Advantages:

- No boxing
- Faster
- Less memory

Example:

```
IntStream.rangeClosed(1,10)
```

Convert:

```
.boxed()
```

10. Infinite Streams

Generate:

```
Stream.generate(Math::random)
```

Iterate:

```
Stream.iterate(1,n->n+1)
```

Always use .limit() — otherwise infinite loop.

11. Lazy Evaluation

Nothing executes until terminal operation.

```
stream
  .filter(...)
  .map(...)
```

→ No execution.

```
.collect(...)
```

→ Execution starts.

12. Short Circuit Operations

- findFirst()
- findAny()
- anyMatch()
- allMatch()
- noneMatch()
- limit()

Stops processing early.

13. Stream Pipeline

```
Source
  ↓
filter
  ↓
map
  ↓
sorted
  ↓
limit
  ↓
collect
```

14. map() vs flatMap()

map()	flatMap()
One → One	One → Many
Employee → Name	Employee → Skills
Returns Stream<T>	Flattens nested Streams

15. groupingBy() vs partitioningBy()

groupingBy	partitioningBy
Multiple groups	Exactly two groups
Key can be anything	Boolean key

16. findFirst() vs findAny()

findFirst	findAny
Ordered	Unordered
Sequential	Parallel friendly

17. forEach() vs forEachOrdered()

Parallel:

```
.parallelStream()
.forEach()
```

Output order is unpredictable.

Ordered:

```
.forEachOrdered()
```

Maintains encounter order.

18. Common Mistakes

❌ Reusing Stream

```
stream.count();
stream.forEach(...);
```

→ IllegalStateException

❌ No Terminal Operation

```
stream.filter(...)
```

Nothing executes.

❌ Using map instead of flatMap

Wrong:

```
Stream<List<String>>
```

Correct:

```
Stream<String>
```

❌ Side Effects

Wrong:

```
list.stream()
  .forEach(result::add)
```

Correct:

```
list.stream()
  .map(...)
  .toList()
```

❌ Parallel Stream + ArrayList

Wrong:

```
parallelStream()
  .forEach(list::add)
```

ArrayList is not thread-safe.

19. Performance Tips

✅ Filter early:

```
.filter(...)
.limit(...)
.map(...)
```

Better than:

```
.map(...)
.filter(...)
```

- Use primitive streams when possible
- Avoid unnecessary sorting
- Prefer method references

Method reference:

```
Employee::getName
```

instead of:

```
e->e.getName()
```

Avoid parallel streams unless needed.

20. Internal Working

```
Collection
  ↓
Stream
  ↓
Spliterator
  ↓
Pipeline
  ↓
Intermediate Operations (Lazy)
  ↓
Terminal Operation
  ↓
Result
```

Parallel Streams

```
Collection
  ↓
Spliterator
  ↓
Split
  ↓
Thread 1   Thread 2   Thread 3
  ↓
Merge
  ↓
Result
```

Uses:

```
ForkJoinPool.commonPool()
```

21. Most Asked Interview Questions

- What is Stream API?
- Difference between Collection and Stream?
- Why are Streams lazy?
- Difference between filter() and map()?
- Difference between map() and flatMap()?
- Difference between groupingBy() and partitioningBy()?
- Difference between findFirst() and findAny()?

- Difference between stream() and parallelStream()?
- How does reduce() work?
- What is Spliterator?
- What is boxing/unboxing?
- Why use IntStream?
- Why can't a Stream be reused?
- Why does toMap() throw Duplicate Key Exception?
- When should parallel streams be avoided?
- What is Optional and why does findFirst() return one?
- Difference between Stream.toList() and Collectors.toList()?
- Difference between stateless and stateful intermediate operations?
- Why can't lambdas throw checked exceptions directly?

22. Real Spring Boot Example

```
List<String> names =
    employees.stream()
             .filter(e -> e.getDepartment().equals("IT"))
             .filter(e -> e.getSalary()>100000)
             .sorted(
                 Comparator.comparing(Employee::getSalary)
                           .reversed()
             )
             .limit(5)
             .map(Employee::getName)
             .toList();
```

Pipeline

```
Employees
  ↓
Filter Department
  ↓
Filter Salary
  ↓
Sort
  ↓
Limit
  ↓
Map
  ↓
List
```

23. Concepts That Fill the Gaps

Things used throughout these notes (Optional, Spliterator, method references) but never fully explained — plus a few pitfalls worth knowing.

Optional — what findFirst()/min()/reduce() actually return

A container that may or may not hold a value. Forces you to handle the "no result" case instead of risking a NullPointerException.

```
Optional<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > 1000000)
    .findFirst();

result.isPresent()          // true / false
result.isEmpty()            // Java 11+
result.get()                // throws if empty — avoid calling blindly
result.orElse(defaultEmp)   // fallback value
result.orElseGet(() -> ...) // fallback computed lazily
result.orElseThrow(...)     // custom exception if empty
result.ifPresent(e -> System.out.println(e.getName()));
```

Rule of thumb: never call .get() without checking .isPresent() first — prefer orElse()/orElseGet()/ifPresent().

Spliterator — the thing behind parallel splitting

A "splittable iterator." Every Stream is backed by one. It defines how a source can be divided into chunks (for parallel streams) and reports characteristics like ORDERED, SIZED, DISTINCT, and SORTED that let the Stream pipeline optimize itself.

```
Spliterator<Integer> sp = list.spliterator();
sp.tryAdvance(System.out::println); // process one element
sp.trySplit();                      // split off a chunk for another thread
```

You rarely call it directly — it's the internal engine that makes .parallelStream() possible.

Method References — the 4 kinds

Kind	Example
Static method	Integer::parseInt
Instance method of a particular object	System.out::println
Instance method of an arbitrary object of a type	String::toUpperCase
Constructor reference	ArrayList::new

```
// Static method — Integer::parseInt
.map(Integer::parseInt)              // same as s -> Integer.parseInt(s)

// Particular object — System.out::println
.forEach(System.out::println)        // same as x -> System.out.println(x)

// Arbitrary object of a type — String::toUpperCase
.map(String::toUpperCase)            // same as s -> s.toUpperCase()

// Constructor reference — ArrayList::new
.collect(Collectors.toCollection(ArrayList::new)) // same as () -> new
ArrayList<>()
```

Stream.toList() vs Collectors.toList()

```
stream.toList()                  // returns an UNMODIFIABLE list (Java 16+)
stream.collect(Collectors.toList()) // returns a MUTABLE ArrayList
(implementation detail)
```

Calling .add() on the result of .toList() throws UnsupportedOperationException — use Collectors.toList() if you need a mutable result.

Stateless vs Stateful Intermediate Operations

Stateless	Stateful
filter(), map(), peek()	sorted(), distinct(), limit(), skip()
Processes each element independently	Needs to see other elements first
Works well with infinite streams	May buffer the whole stream (sorted/distinct)

This is why Stream.iterate(1, n -> n+1).sorted() never terminates — sorted() must see every element, but the stream is infinite.

```
// Stateless — each element handled independently, prints as it goes
Stream.of(3, 1, 2)
    .peek(n -> System.out.println("filtering " + n))
    .filter(n -> n > 1)
    .forEach(System.out::println);

// Stateful — sorted() must buffer ALL elements before it can emit the first one
Stream.of(3, 1, 2)
    .sorted()                  // waits for the whole stream, THEN sorts
    .forEach(System.out::println);   // 1, 2, 3

// Danger: this hangs forever — sorted() never gets to see "all" elements
// Stream.iterate(1, n -> n + 1).sorted().findFirst();
```

Checked Exceptions Inside Lambdas

Functional interfaces like Function and Predicate don't declare checked exceptions, so this fails to compile:

```
// Won't compile if readFile throws IOException
.map(file -> readFile(file))
```

Fix: wrap in try/catch inside the lambda, or rethrow as an unchecked exception.

```
.map(file -> {
    try {
        return readFile(file);
    } catch (IOException e) {
        throw new UncheckedIOException(e);
    }
})
```

Comparator Chaining

```
Comparator.comparing(Employee::getDepartment)
```

```
            .thenComparing(Employee::getSalary, Comparator.reverseOrder())
            .thenComparing(Employee::getName)
```

Reads as: sort by department, then by salary (descending) within each department, then by name as a final tie-breaker.

A Few More Handy Methods

- takeWhile(predicate) — Java 9+, takes elements while true then stops (short-circuits, unlike filter)
- dropWhile(predicate) — Java 9+, opposite of takeWhile
- Stream.ofNullable(value) — Java 9+, empty stream if value is null, single-element stream otherwise
- IntStream.summaryStatistics() — count, sum, min, max, average in one pass
- Collectors.teeing(...) — Java 12+, runs two collectors and merges their results

takeWhile() vs filter()

```
List.of(1, 2, 3, 4, 1, 2)
    .stream()
    .takeWhile(n -> n < 4)
    .forEach(System.out::println);
// Output: 1, 2, 3  — STOPS at the first failure (4), ignores the later 1, 2

List.of(1, 2, 3, 4, 1, 2)
    .stream()
    .filter(n -> n < 4)
    .forEach(System.out::println);
// Output: 1, 2, 3, 1, 2  — checks EVERY element, keeps all matches
```

dropWhile()

```
List.of(1, 2, 3, 4, 1, 2)
    .stream()
    .dropWhile(n -> n < 4)
    .forEach(System.out::println);
// Output: 4, 1, 2  — drops elements UNTIL the condition first fails, keeps the
rest
```

Stream.ofNullable()

```
String value = null;

long count = Stream.ofNullable(value).count();
// count == 0, no NullPointerException

Stream.ofNullable("hello").forEach(System.out::println);
// prints: hello

// Handy combined with flatMap on a list that may contain nulls:
list.stream()
    .flatMap(Stream::ofNullable)
    .toList(); // nulls silently skipped
```

IntStream.summaryStatistics()

```
var stats = employees.stream()
```

```
    .mapToInt(Employee::getSalary)
    .summaryStatistics();

stats.getCount();    // number of employees
stats.getSum();      // total salary
stats.getMin();      // lowest salary
stats.getMax();      // highest salary
stats.getAverage();  // average salary
```

Collectors.teeing()

```
// Compute average salary WITHOUT two separate passes over the stream
double average = employees.stream()
    .collect(Collectors.teeing(
        Collectors.summingDouble(Employee::getSalary), // first collector
        Collectors.counting(),                          // second collector
        (sum, count) -> sum / count                     // merge function
    ));
```

24. Quick Revision Cheat Sheet

Create

- stream()
- parallelStream()
- Stream.of()
- Arrays.stream()

Intermediate

- filter()
- map()
- flatMap()
- sorted()
- distinct()
- peek()
- skip()
- limit()

Terminal

- forEach()
- collect()
- toList()
- count()
- reduce()
- min()
- max()

- findFirst()
- findAny()
- anyMatch()
- allMatch()
- noneMatch()

Collectors

- toList()
- toSet()
- toMap()
- joining()
- groupingBy()
- partitioningBy()
- mapping()
- counting()
- averagingDouble()
- summingDouble()
- summarizingInt()
- maxBy()
- minBy()

Primitive Streams

- IntStream
- LongStream
- DoubleStream

Advanced

- Lazy Evaluation
- Short Circuiting
- Parallel Stream
- ForkJoinPool
- Spliterator
- Pipeline
- Thread Safety
- Performance Optimization
- Optional
- Method References
- Stateless vs Stateful Ops
- takeWhile() / dropWhile()
- Collectors.teeing()

These notes are sufficient for:

- Java Stream fundamentals
- Spring Boot development
- 3–8 years Java interview preparation
- Daily development reference
- Most Stream-related coding interview questions